

IT012-Lab04

Họ và tên: Nguyễn Hải Phong

MSSV: 25521383

1. Lý thuyết

Giảng viên hướng dẫn sinh viên về chương trình hợp ngữ MIPS dựa theo tài liệu: **Tổng quát về hợp ngữ và kiến trúc MIPS**

2. Thực hành

Chuyển đoạn code trong bảng theo sau sang MIPS và sử dụng MARS để kiểm tra lại kết quả:

```
if (i == j)
    f = g + h;
else
    f = g - h;
```

(Với giá trị của i, j, f, g, h lần lượt chứa trong RAM)

Code sau khi chỉnh sang định dạng MIPS:

```
.data
i: .word 5
j: .word 5
g: .word 10
h: .word 2
f: .word 0
.text
la $t0, i
lw $s0, 0($t0)
la $t0, j
lw $s1, 0($t0)
la $t0, f
lw $s2, 0($t0)
la $t0, g
lw $s3, 0($t0)
```

```

la $t0, h

lw $s4, 0($t0)

beq $s0, $s1, IF

sub $s2, $s3, $s4

IF:

    add $s2, $s3, $s4

la $t0, f

sw $s2, 0($t0)

```

Mô phỏng quá trình chạy trong MARS:

RAM khi khởi tạo

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)
0x10010000	0x00000005	0x00000005	0x0000000a	0x00000002

Thanh ghi sau khi load word

\$zero	0	0x00000000
\$at	1	0x10010000
\$v0	2	0x00000000
\$v1	3	0x00000000
\$a0	4	0x00000000
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x1001000c
\$t1	9	0x00000000
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$s0	16	0x00000005
\$s1	17	0x00000005
\$s2	18	0x00000000
\$s3	19	0x0000000a
\$s4	20	0x00000002

Sau khi thực hiện lệnh beq với các giá trị khởi tạo là i = 5, j = 5, f = 0, g = 10, h = 2

\$s0	16	0x00000005
\$s1	17	0x00000005
\$s2	18	0x0000000c
\$s3	19	0x0000000a
\$s4	20	0x00000002

Thanh RAM sau khi ghi lại biến f

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)
0x10010000	0x00000005	0x00000005	0x0000000a	0x00000002	0x0000000c

```

int Sum = 0
for (int i = 1; i <= N; ++i){
    Sum = Sum + i;
}

```

(Với giá trị của i, N, Sum lần lượt chứa trong RAM)

Code sau khi chỉnh sang định dạng MIPS:

.data

Sum: .word 0

N : .word 5

i : .word 1

.text

la \$t0, Sum

lw \$s0, 0(\$t0)

la \$t0, N

lw \$s1, 0(\$t0)

la \$t0, i

lw \$s2, 0(\$t0)

LOOP:

slt \$t1, \$s2, \$s1

beq \$t1, \$zero, EXIT

add \$s0, \$s0, \$s2

addi \$s2, \$s2, 1

j LOOP

EXIT:

add \$s0, \$s0, \$s1

la \$t0, Sum

sw \$s0, 0(\$t0)

la \$t0, i

sw \$s2, 0(\$t0)

Mô phỏng quá trình chạy trong MARS:

RAM khi khởi tạo

Data Segment			
Address	Value (+0)	Value (+4)	Value (+8)
0x10010000	0x00000000	0x00000005	0x00000001

Thanh ghi sau khi load word	<table><tr><th>Name</th><th>Number</th><th>Value</th></tr><tr><td>\$zero</td><td>0</td><td>0x00000000</td></tr><tr><td>\$at</td><td>1</td><td>0x10010000</td></tr><tr><td>\$v0</td><td>2</td><td>0x00000000</td></tr><tr><td>\$v1</td><td>3</td><td>0x00000000</td></tr><tr><td>\$a0</td><td>4</td><td>0x00000000</td></tr><tr><td>\$a1</td><td>5</td><td>0x00000000</td></tr><tr><td>\$a2</td><td>6</td><td>0x00000000</td></tr><tr><td>\$a3</td><td>7</td><td>0x00000000</td></tr><tr><td>\$t0</td><td>8</td><td>0x10010008</td></tr><tr><td>\$t1</td><td>9</td><td>0x00000000</td></tr><tr><td>\$t2</td><td>10</td><td>0x00000000</td></tr><tr><td>\$t3</td><td>11</td><td>0x00000000</td></tr><tr><td>\$t4</td><td>12</td><td>0x00000000</td></tr><tr><td>\$t5</td><td>13</td><td>0x00000000</td></tr><tr><td>\$t6</td><td>14</td><td>0x00000000</td></tr><tr><td>\$t7</td><td>15</td><td>0x00000000</td></tr><tr><td>\$s0</td><td>16</td><td>0x00000000</td></tr><tr><td>\$s1</td><td>17</td><td>0x00000005</td></tr><tr><td>\$s2</td><td>18</td><td>0x00000001</td></tr></table>	Name	Number	Value	\$zero	0	0x00000000	\$at	1	0x10010000	\$v0	2	0x00000000	\$v1	3	0x00000000	\$a0	4	0x00000000	\$a1	5	0x00000000	\$a2	6	0x00000000	\$a3	7	0x00000000	\$t0	8	0x10010008	\$t1	9	0x00000000	\$t2	10	0x00000000	\$t3	11	0x00000000	\$t4	12	0x00000000	\$t5	13	0x00000000	\$t6	14	0x00000000	\$t7	15	0x00000000	\$s0	16	0x00000000	\$s1	17	0x00000005	\$s2	18	0x00000001
Name	Number	Value																																																											
\$zero	0	0x00000000																																																											
\$at	1	0x10010000																																																											
\$v0	2	0x00000000																																																											
\$v1	3	0x00000000																																																											
\$a0	4	0x00000000																																																											
\$a1	5	0x00000000																																																											
\$a2	6	0x00000000																																																											
\$a3	7	0x00000000																																																											
\$t0	8	0x10010008																																																											
\$t1	9	0x00000000																																																											
\$t2	10	0x00000000																																																											
\$t3	11	0x00000000																																																											
\$t4	12	0x00000000																																																											
\$t5	13	0x00000000																																																											
\$t6	14	0x00000000																																																											
\$t7	15	0x00000000																																																											
\$s0	16	0x00000000																																																											
\$s1	17	0x00000005																																																											
\$s2	18	0x00000001																																																											
Thanh ghi sau khi kết thúc vòng lặp và cộng và giá trị N	<table><tr><td>\$s0</td><td>16</td><td>0x0000000f</td></tr><tr><td>\$s1</td><td>17</td><td>0x00000005</td></tr><tr><td>\$s2</td><td>18</td><td>0x00000005</td></tr></table>	\$s0	16	0x0000000f	\$s1	17	0x00000005	\$s2	18	0x00000005																																																			
\$s0	16	0x0000000f																																																											
\$s1	17	0x00000005																																																											
\$s2	18	0x00000005																																																											
Thanh RAM sau khi store word biến i và biến Sum và lại RAM	<table><tr><th colspan="4">Data Segment</th></tr><tr><th>Address</th><th>Value (+0)</th><th>Value (+4)</th><th>Value (+8)</th></tr><tr><td>0x10010000</td><td>0x0000000f</td><td>0x00000005</td><td>0x00000001</td></tr></table>	Data Segment				Address	Value (+0)	Value (+4)	Value (+8)	0x10010000	0x0000000f	0x00000005	0x00000001																																																
Data Segment																																																													
Address	Value (+0)	Value (+4)	Value (+8)																																																										
0x10010000	0x0000000f	0x00000005	0x00000001																																																										

3. Bài tập

a. Nhập vào một ký tự, xuất ra cửa sổ I/O của MARS theo từng yêu cầu sau:

✓ Ký tự liền trước và liền sau của ký tự nhập vào

Ví dụ:

Nhap ky tu (chỉ một ký tự): b

Ky tu truoc: a

Ky tu sau: c

✓ Ký tự nhập vào chỉ được phép là ba loại: số, chữ thường và chữ hoa. Nếu ký tự nhập vào rơi vào một trong ba loại, xuất ra cửa sổ đó là loại nào; nếu ký tự nhập không rơi vào một trong ba loại trên, xuất ra thông báo “invalid type”

Kết quả các lần chạy bao gồm nhập số, chữ thường chữ hoa và trường hợp invalid type:

```

Nhap ky tu (chi mot ky tu):v
Ky tu truoc: u
Ky tu sau: w
-- program is finished running --

Nhap ky tu (chi mot ky tu):5
Ky tu truoc: 4
Ky tu sau: 6
-- program is finished running --

Nhap ky tu (chi mot ky tu):B
Ky tu truoc: A
Ky tu sau: C
-- program is finished running --

Nhap ky tu (chi mot ky tu):;
invalid type
-- program is finished running (dropped off bottom) --

```

Code assembly:

```

.data
    msg_input: .asciiz "Nhap ky tu (chi mot ky
tu):"
    msg_before: .asciiz "\n Ky tu truoc: "
    msg_after: .asciiz "\n Ky tu sau: "
    msg_invalid: .asciiz "\n invalid type"
.text
main:
    li, $v0, 4
    la $a0, msg_input
    syscall
    li $v0, 12
    syscall
    move $s0, $v0
CHECK_NUM:
    li $t1, '0'
    li $t2, '9'
    blt $s0, $t1, INVALID
    ble $s0, $t2, VALID
CHECK_UPPER:

```

```
li $t1, 'A'
li $t2, 'Z'
blt $s0, $t1, INVALID
ble $s0, $t2, VALID
```

CHECK_LOWER:

```
li $t1, 'a'
li $t2, 'z'
blt $s0, $t1, INVALID
ble $s0, $t2, VALID
```

VALID:

```
li $v0, 4
la $a0, msg_before
syscall
addi $a0, $s0, -1
li $v0, 11
syscall
li $v0, 4
la $a0, msg_after
syscall
addi $a0, $s0, 1
li $v0, 11
syscall
li $v0, 10
syscall
```

INVALID:

```
li $v0, 4
la $a0, msg_invalid
syscall
```

b. Nhập từ bàn phím 2 số nguyên, in ra cửa sổ I/O của MARS theo từng yêu cầu sau:

- ✓ Số lớn hơn
- ✓ Tổng, hiệu, tích và thương của hai số

Kết quả màn hình I / O khi thực hiện xong:

```
Nhap so thu nhat: 5
Nhap so thu hai: 3

So nguyen lon hon: 5
Tong 2 so: 8
Hieu 2 so: 2
Tich 2 so: 15
Thuong 2 so: 1
-- program is finished running (dropped off bottom) --
```

Code assembly:

```
.data

msg_input_1: .asciiz "Nhap so thu nhat: "
msg_input_2: .asciiz "\nNhap so thu hai: "
msg_print_larger: .asciiz "\nSo nguyen lon hon: "
msg_sum: .asciiz "\nTong 2 so: "
msg_sub: .asciiz "\nHieu 2 so: "
msg_product: .asciiz "\nTich 2 so: "
msg_divide: .asciiz "\nThuong 2 so: "

.text
main:
    li $v0, 4
    la $a0, msg_input_1
    syscall
    li $v0, 5
    syscall
    move $s0, $v0
    li $v0, 4
    la $a0, msg_input_2
    syscall
    li $v0, 5
    syscall
```

move \$s1, \$v0

CHECK_LARGER:

li, \$v0, 4

la \$a0, msg_print_larger

syscall

slt \$t0, \$s0, \$s1

beq \$t0, \$zero, IF

li \$v0, 1

move \$a0, \$s1

syscall

IF:

li \$v0, 1

move \$a0, \$s0

syscall

PRINT_SUM:

li \$v0, 4

la \$a0, msg_sum

syscall

li \$v0, 1

add \$a0, \$s0, \$s1

syscall

PRINT_SUB:

li \$v0, 4

la \$a0, msg_sub

syscall

li \$v0, 1

sub \$a0, \$s0, \$s1

syscall

PRINT_PRODUCT:

li \$v0, 4

la \$a0, msg_product

syscall

li \$v0, 1


```

mul $a0, $s0, $s1
syscall
PRINT_DIV:
li $v0, 4
la $a0, msg_divide
syscall
li $v0, 1
div $s0, $s1
move $a0 $s0
mflo $a0
syscall

```

- c. Viết chương trình cho người dùng nhập vào số n có 2 chữ số và tính số Fibonacci thứ n

Kết quả chương trình:

```

Nhap so nguyen co 2 chu so: 10

So fibonacci thu n: 55
-- program is finished running (dropped off bottom) --

```

Code assembly:

```

.data
msg_input: .asciiz "Nhap so nguyen co 2 chu so: "
msg_output: .asciiz "\nSo fibonacci thu n: "
.text
li $v0, 4
la $a0, msg_input
syscall

```

li \$v0, 5

syscall

move \$s0, \$v0

li \$t0, 0

li \$t1, 1

li \$t2, 2

LOOP:

add \$t3, \$t0, \$t1

move \$t0, \$t1

move \$t1, \$t3

beq \$s0, \$t2 PRINT_RESULT

addi \$t2, \$t2, 1

j LOOP

PRINT_RESULT:

li \$v0, 4

la \$a0, msg_output

syscall

li \$v0, 1

move \$a0, \$t1

syscall